

HW 3

Dev Gandhi

2026-06-25

Set your conflict_prefer.

```
conflict_prefer("select", "dplyr")
```

```
## [conflicted] Removing existing preference.  
## [conflicted] Will prefer dplyr::select over any other package.
```

```
conflict_prefer("tune", "tune")
```

```
## [conflicted] Removing existing preference.  
## [conflicted] Will prefer tune::tune over any other package.
```

```
conflict_prefer("chisq.test", "stats")
```

```
## [conflicted] Will prefer stats::chisq.test over any other package.
```

```
conflict_prefer("filter", "dplyr")
```

```
## [conflicted] Removing existing preference.  
## [conflicted] Will prefer dplyr::filter over any other package.
```

```
conflict_prefer("fit", "parsnip")
```

```
## [conflicted] Will prefer parsnip::fit over any other package.
```

```
conflict_prefer("rmse", "yardstick")
```

```
## [conflicted] Will prefer yardstick::rmse over any other package.
```

```
conflict_prefer("map", "purrr")
```

```
## [conflicted] Will prefer purrr::map over any other package.
```

```
conflict_prefer("vi", "vip")
```

```
## [conflicted] Will prefer vip::vi over any other package.
```

```
conflict_prefer("skewness", "PerformanceAnalytics")
```

```
## [conflicted] Will prefer PerformanceAnalytics::skewness over any other package.
```

```
conflict_prefer("accuracy", "yardstick")
```

```
## [conflicted] Will prefer yardstick::accuracy over any other package.
```

```
# From Matt Dancho DS4B 201 - ggpairs wrapper with optional color grouping  
plot_ggpairs <- function(data, color = NULL, density_alpha = 0.5) {
```

```
  color_expr <- enquos(color)
```

```

if (rlang::quo_is_null(color_expr)) {
  g <- data %>% ggpairs(lower = "blank")
} else {
  color_name <- quo_name(color_expr)
  g <- data %>%
    ggpairs(
      mapping = aes(color = .data[[color_name]]),
      lower = "blank",
      legend = 1,
      diag = list(continuous = wrap("densityDiag", alpha = density_alpha))
    ) +
    theme(legend.position = "bottom")
}
return(g)
}

# From Matt Dancho DS4B 201 - faceted histogram of all variables
plot_hist_facet <- function(data, fct_reorder = FALSE, fct_rev = FALSE,
                             bins = 10, fill = palette_light()[[3]],
                             color = "white", ncol = 5, scale = "free") {

  # pivot_longer replaces the deprecated gather() call from the original
  data_factored <- data %>%
    mutate(across(where(is.character), as.factor)) %>%
    mutate(across(where(is.factor), as.numeric)) %>%
    pivot_longer(everything(), names_to = "key", values_to = "value")

  if (fct_reorder) {
    data_factored <- data_factored %>%
      mutate(key = as.character(key) %>% as.factor())
  }

  if (fct_rev) {
    data_factored <- data_factored %>% mutate(key = fct_rev(key))
  }

  g <- data_factored %>%
    ggplot(aes(x = value, group = key)) +
    geom_histogram(bins = bins, fill = fill, color = color) +
    facet_wrap(~ key, ncol = ncol, scale = scale) +
    theme_tq()

  return(g)
}

```

Bring in the data. This is the IBM HR data with 1470 observations we used before.

```

# EDIT THIS PATH
data_path <- "~/Desktop/UGA IOMP/Summer '26/Advanced Analytics/Advanced Analytics R/WA_Fn-UseC_-HR-Emplo
#

df <- read_excel(data_path)
colnames(df)

```

```
## [1] "Age" "Attrition"
```

```
## [3] "BusinessTravel"      "DailyRate"
## [5] "Department"          "DistanceFromHome"
## [7] "Education"           "EducationField"
## [9] "EmployeeCount"       "EmployeeNumber"
## [11] "EnvironmentSatisfaction" "Gender"
## [13] "HourlyRate"          "JobInvolvement"
## [15] "JobLevel"            "JobRole"
## [17] "JobSatisfaction"     "MaritalStatus"
## [19] "MonthlyIncome"       "MonthlyRate"
## [21] "NumCompaniesWorked"  "Over18"
## [23] "OverTime"            "PercentSalaryHike"
## [25] "PerformanceRating"   "RelationshipSatisfaction"
## [27] "StandardHours"       "StockOptionLevel"
## [29] "TotalWorkingYears"   "TrainingTimesLastYear"
## [31] "WorkLifeBalance"     "YearsAtCompany"
## [33] "YearsInCurrentRole"  "YearsSinceLastPromotion"
## [35] "YearsWithCurrManager"
```

```
str(df)
```

```
## tibble [1,470 x 35] (S3: tbl_df/tbl/data.frame)
## $ Age : num [1:1470] 41 49 37 33 27 32 59 30 38 36 ...
## $ Attrition : chr [1:1470] "Yes" "No" "Yes" "No" ...
## $ BusinessTravel : chr [1:1470] "Travel_Rarely" "Travel_Frequently" "Travel_Rarely" "Travel_Frequently" ...
## $ DailyRate : num [1:1470] 1102 279 1373 1392 591 ...
## $ Department : chr [1:1470] "Sales" "Research & Development" "Research & Development" "Sales" ...
## $ DistanceFromHome : num [1:1470] 1 8 2 3 2 2 3 24 23 27 ...
## $ Education : num [1:1470] 2 1 2 4 1 2 3 1 3 3 ...
## $ EducationField : chr [1:1470] "Life Sciences" "Life Sciences" "Other" "Life Sciences" ...
## $ EmployeeCount : num [1:1470] 1 1 1 1 1 1 1 1 1 1 ...
## $ EmployeeNumber : num [1:1470] 1 2 4 5 7 8 10 11 12 13 ...
## $ EnvironmentSatisfaction : num [1:1470] 2 3 4 4 1 4 3 4 4 3 ...
## $ Gender : chr [1:1470] "Female" "Male" "Male" "Female" ...
## $ HourlyRate : num [1:1470] 94 61 92 56 40 79 81 67 44 94 ...
## $ JobInvolvement : num [1:1470] 3 2 2 3 3 3 4 3 2 3 ...
## $ JobLevel : num [1:1470] 2 2 1 1 1 1 1 1 3 2 ...
## $ JobRole : chr [1:1470] "Sales Executive" "Research Scientist" "Laboratory Technician" "Sales Executive" ...
## $ JobSatisfaction : num [1:1470] 4 2 3 3 2 4 1 3 3 3 ...
## $ MaritalStatus : chr [1:1470] "Single" "Married" "Single" "Married" ...
## $ MonthlyIncome : num [1:1470] 5993 5130 2090 2909 3468 ...
## $ MonthlyRate : num [1:1470] 19479 24907 2396 23159 16632 ...
## $ NumCompaniesWorked : num [1:1470] 8 1 6 1 9 0 4 1 0 6 ...
## $ Over18 : chr [1:1470] "Y" "Y" "Y" "Y" ...
## $ OverTime : chr [1:1470] "Yes" "No" "Yes" "Yes" ...
## $ PercentSalaryHike : num [1:1470] 11 23 15 11 12 13 20 22 21 13 ...
## $ PerformanceRating : num [1:1470] 3 4 3 3 3 3 4 4 4 3 ...
## $ RelationshipSatisfaction : num [1:1470] 1 4 2 3 4 3 1 2 2 2 ...
## $ StandardHours : num [1:1470] 80 80 80 80 80 80 80 80 80 80 ...
## $ StockOptionLevel : num [1:1470] 0 1 0 0 1 0 3 1 0 2 ...
## $ TotalWorkingYears : num [1:1470] 8 10 7 8 6 8 12 1 10 17 ...
## $ TrainingTimesLastYear : num [1:1470] 0 3 3 3 3 2 3 2 2 3 ...
## $ WorkLifeBalance : num [1:1470] 1 3 3 3 3 2 2 3 3 2 ...
## $ YearsAtCompany : num [1:1470] 6 10 0 8 2 7 1 1 9 7 ...
## $ YearsInCurrentRole : num [1:1470] 4 7 0 7 2 7 0 0 7 7 ...
## $ YearsSinceLastPromotion : num [1:1470] 0 1 0 3 2 3 0 0 1 7 ...
```

```
## $ YearsWithCurrManager      : num [1:1470] 5 7 0 0 2 6 0 0 8 7 ...

# Convert all character columns to factors
# mutate(across(where(is.character), as.factor)) is the modern replacement
# for as.data.frame(unclass(df)) which works but is harder to read
df <- df %>%
  mutate(across(where(is.character), as.factor))

## mutate: converted 'Attrition' from character to factor (0 new NA)
##      converted 'BusinessTravel' from character to factor (0 new NA)
##      converted 'Department' from character to factor (0 new NA)
##      converted 'EducationField' from character to factor (0 new NA)
##      converted 'Gender' from character to factor (0 new NA)
##      converted 'JobRole' from character to factor (0 new NA)
##      converted 'MaritalStatus' from character to factor (0 new NA)
##      converted 'Over18' from character to factor (0 new NA)
##      converted 'OverTime' from character to factor (0 new NA)

# Explicitly call out the level needed for attrition so that "Yes" is the reference
df <- df %>%
  mutate(Attrition = relevel(factor(Attrition), ref = "Yes"))

## mutate: changed 0 values (0%) of 'Attrition' (0 new NAs)
str(df)

## tibble [1,470 x 35] (S3: tbl_df/tbl/data.frame)
## $ Age                : num [1:1470] 41 49 37 33 27 32 59 30 38 36 ...
## $ Attrition           : Factor w/ 2 levels "Yes","No": 1 2 1 2 2 2 2 2 2 ...
## $ BusinessTravel      : Factor w/ 3 levels "Non-Travel","Travel_Frequently",...: 3 2 3 2 3 2 3 3 ...
## $ DailyRate           : num [1:1470] 1102 279 1373 1392 591 ...
## $ Department          : Factor w/ 3 levels "Human Resources",...: 3 2 2 2 2 2 2 2 2 ...
## $ DistanceFromHome    : num [1:1470] 1 8 2 3 2 2 3 24 23 27 ...
## $ Education           : num [1:1470] 2 1 2 4 1 2 3 1 3 3 ...
## $ EducationField      : Factor w/ 6 levels "Human Resources",...: 2 2 5 2 4 2 4 2 2 4 ...
## $ EmployeeCount       : num [1:1470] 1 1 1 1 1 1 1 1 1 1 ...
## $ EmployeeNumber      : num [1:1470] 1 2 4 5 7 8 10 11 12 13 ...
## $ EnvironmentSatisfaction : num [1:1470] 2 3 4 4 1 4 3 4 4 3 ...
## $ Gender              : Factor w/ 2 levels "Female","Male": 1 2 2 1 2 2 1 2 2 2 ...
## $ HourlyRate          : num [1:1470] 94 61 92 56 40 79 81 67 44 94 ...
## $ JobInvolvement      : num [1:1470] 3 2 2 3 3 3 4 3 2 3 ...
## $ JobLevel            : num [1:1470] 2 2 1 1 1 1 1 1 3 2 ...
## $ JobRole             : Factor w/ 9 levels "Healthcare Representative",...: 8 7 3 7 3 3 3 3 5 1 ...
## $ JobSatisfaction     : num [1:1470] 4 2 3 3 2 4 1 3 3 3 ...
## $ MaritalStatus       : Factor w/ 3 levels "Divorced","Married",...: 3 2 3 2 2 3 2 1 3 2 ...
## $ MonthlyIncome       : num [1:1470] 5993 5130 2090 2909 3468 ...
## $ MonthlyRate         : num [1:1470] 19479 24907 2396 23159 16632 ...
## $ NumCompaniesWorked  : num [1:1470] 8 1 6 1 9 0 4 1 0 6 ...
## $ Over18              : Factor w/ 1 level "Y": 1 1 1 1 1 1 1 1 1 1 ...
## $ OverTime            : Factor w/ 2 levels "No","Yes": 2 1 2 2 1 1 2 1 1 1 ...
## $ PercentSalaryHike   : num [1:1470] 11 23 15 11 12 13 20 22 21 13 ...
## $ PerformanceRating   : num [1:1470] 3 4 3 3 3 3 4 4 3 ...
## $ RelationshipSatisfaction: num [1:1470] 1 4 2 3 4 3 1 2 2 2 ...
## $ StandardHours       : num [1:1470] 80 80 80 80 80 80 80 80 80 80 ...
## $ StockOptionLevel    : num [1:1470] 0 1 0 0 1 0 3 1 0 2 ...
## $ TotalWorkingYears   : num [1:1470] 8 10 7 8 6 8 12 1 10 17 ...
```

```
## $ TrainingTimesLastYear : num [1:1470] 0 3 3 3 3 2 3 2 2 3 ...
## $ WorkLifeBalance       : num [1:1470] 1 3 3 3 3 2 2 3 3 2 ...
## $ YearsAtCompany        : num [1:1470] 6 10 0 8 2 7 1 1 9 7 ...
## $ YearsInCurrentRole    : num [1:1470] 4 7 0 7 2 7 0 0 7 7 ...
## $ YearsSinceLastPromotion : num [1:1470] 0 1 0 3 2 3 0 0 1 7 ...
## $ YearsWithCurrManager  : num [1:1470] 5 7 0 0 2 6 0 0 8 7 ...
```

#I am adding an id variable before all else

```
df <- df %>%
  mutate(id = row_number()) %>%
  select(id, everything())
```

mutate: new variable 'id' (integer) with 1,470 unique values and 0% NA

*# clean_names() from {janitor} converts all column names to snake_case,
removes special characters, and ensures unique names.
This must run before any column-name references downstream.*

```
df <- df %>%
  clean_names()

cat(colnames(df), sep = ",\n")
```

```
## id,
## age,
## attrition,
## business_travel,
## daily_rate,
## department,
## distance_from_home,
## education,
## education_field,
## employee_count,
## employee_number,
## environment_satisfaction,
## gender,
## hourly_rate,
## job_involvement,
## job_level,
## job_role,
## job_satisfaction,
## marital_status,
## monthly_income,
## monthly_rate,
## num_companies_worked,
## over18,
## over_time,
## percent_salary_hike,
## performance_rating,
## relationship_satisfaction,
## standard_hours,
## stock_option_level,
## total_working_years,
## training_times_last_year,
## work_life_balance,
```

```
## years_at_company,
## years_in_current_role,
## years_since_last_promotion,
## years_with_curr_manager
```

```
library(tidymodels)
df <- as_tibble(df) %>%
  filter(!is.na(attrition))
```

- d. Run model on training data.
- e. Run model on data_split (or whatever you called your Data after splitting it (since tidymodels knows to now run it on the “test” portion).

```
# Set a random seed to ensure reproducibility.
set.seed(2020)
```

```
# I will first start by splitting the Data. Split the dataset into 80% training and 20% testing using initial_split()
#Split the dataset into 80% training and 20% testing using initial_split().
```

```
attrition_split <- initial_split(df, prop = 0.80, strata = "attrition")
attrition_train <- training(attrition_split)
attrition_test <- testing(attrition_split)
```

```
# Create 10 folds for Cross-Validation (CV) on the training data.
# When we test different penalty values later, CV splits the training data 10 more times. It trains on
```

```
set.seed(2020)
attrition_folds <- vfold_cv(attrition_train, v = 10, strata = "attrition")
```

- b. What did you use for feature selection and why? Boruta or just the model? >For feature selection, I relied directly on the model rather than using an external wrapper algorithm like Boruta. Because I utilized a Ridge regression model, the model inherently handles multicollinearity and feature importance by penalizing the coefficients of less impactful variables, shrinking them toward zero. This allows the model to retain the holistic context of all HR variables while mathematically mitigating the risk of overfitting.
- c. Used some step (weights or sampling) to deal with class imbalance.
- d. I used a (lasso, ridge, elastic net) model because (model performance, explainability, etc.). >I used a Ridge regression model because it balances model performance with comprehensive explainability. In an HR analytics context, stakeholders often want to know how every available metric impacts turnover. Unlike Lasso, which would force some variables completely out of the equation, Ridge keeps all predictors in the model but limits the influence of noisy data, giving us a complete but mathematically stable picture of attrition drivers.

```
#I will be running a Ridge Regression. Create a preprocessing "recipe" that defines the target variable
```

```
ridge_recipe <- recipe(attrition ~ ., data = attrition_train) %>%
  update_role(id, new_role = "ID") %>% # Keeps ID in the data but stops it from predicting
  step_rm(any_of(c("over18", "over_18", "employee_count", "standard_hours"))) %>%
  step_dummy(all_nominal_predictors()) %>%
  step_zv(all_predictors()) %>%
  step_normalize(all_predictors()) %>%
  step_smote(attrition)
```

- a. Chose correct mode of “classification” or “regression” for set_mode()

#I will use the `glmnet` package to perform ridge regression. Define a logistic regression model, setting the mode to classification.

```
ridge_spec <- logistic_reg(penalty = tune(), mixture = 0) %>%  
  set_mode("classification") %>%  
  set_engine("glmnet")
```

#Combine the recipe and the model specification into a single workflow. Workflows bundle your preprocessing steps and model specification.

```
ridge_workflow <- workflow() %>%  
  add_recipe(ridge_recipe) %>%  
  add_model(ridge_spec)
```

- i. I used (metric(s)) to choose my model because _____. >I used roc_auc (Area Under the Receiver Operating Characteristic Curve) to choose my optimal penalty value because it is a premier classification metric that evaluates the model's ability to distinguish between our two classes (employees who stay vs. employees who leave). Unlike pure accuracy, which can be easily skewed by our imbalanced dataset, roc_auc accounts for both sensitivity and specificity across all classification thresholds.

#Create a grid of 50 evenly spaced penalty values to test. We don't know how much we need to "shrink" our coefficients, so we'll test a range of values.

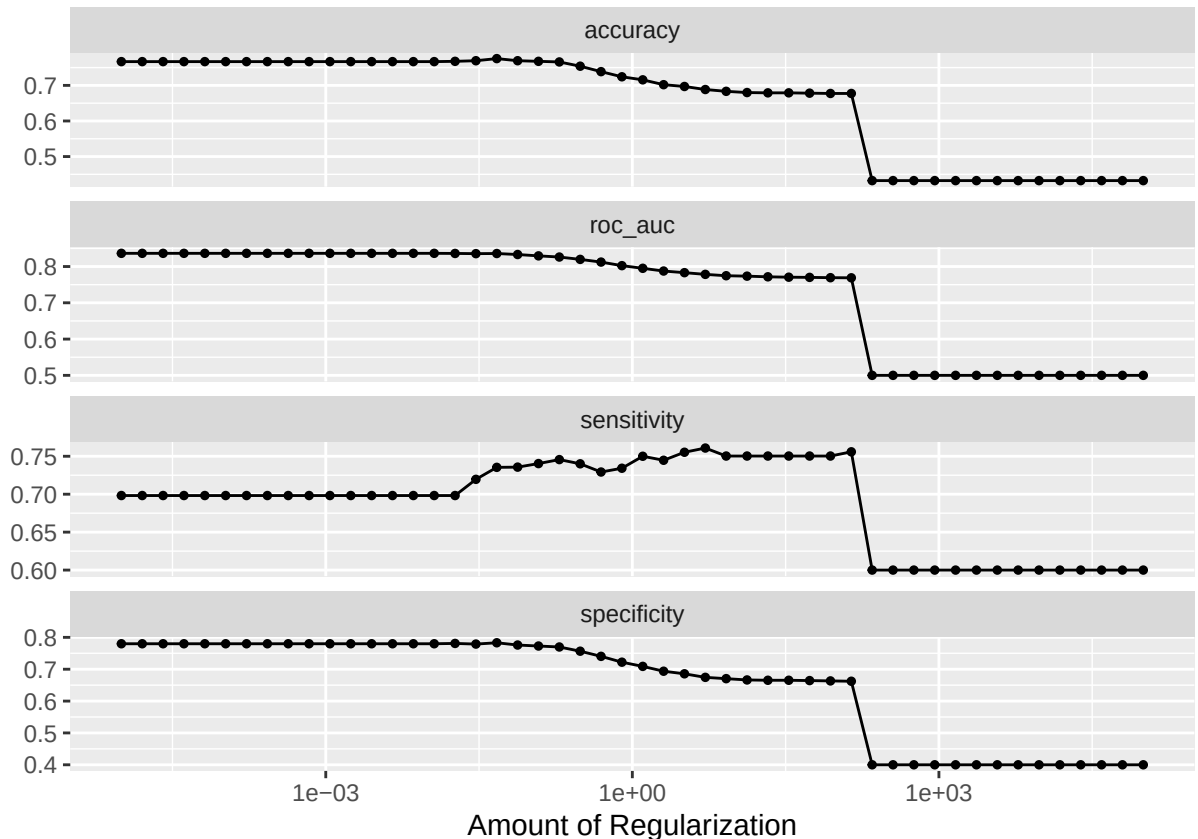
```
penalty_grid <- grid_regular(penalty(range = c(-5, 5)), levels = 50)
```

#Run the tuning process across all 10 CV folds using the 50 penalty values. This creates 500 different model fits.

```
tune_res <- tune_grid(  
  ridge_workflow,  
  resamples = attrition_folds,  
  grid = penalty_grid,  
  metrics = metric_set(roc_auc, accuracy, sensitivity, specificity)  
)
```

#Visualize the performance of the 50 different penalties. Allows us to visually confirm that our penalty values are well-tuned.

```
autoplot(tune_res)
```



- f. Creation and interpretation of a confusion matrix. > The confusion matrix evaluates the model's performance on the 20% unseen testing data. The top-left to bottom-right diagonal represents our correct predictions: True Positives (correctly identifying employees who actually quit) and True Negatives (correctly identifying those who stayed). The off-diagonal numbers represent our errors: False Positives (predicting someone would quit, but they stayed) and False Negatives (predicting someone would stay, but they actually quit). By evaluating these ratios, we can see if the `step_smote` function successfully helped the model identify the minority 'Yes' class.

```
#Extract the single best penalty value based on the highest ROC AUC score. ROC AUC is the gold standard
best_penalty <- select_best(tune_res, metric = "roc_auc")

#Plug that best penalty value back into our placeholder in the workflow. This swaps out the 'tune()' function
ridge_final <- finalize_workflow(ridge_workflow, best_penalty)

#Run last_fit() using the finalized workflow and our initial data split. last_fit() is a convenience function
ridge_final_fit <- last_fit(
  ridge_final,
  split = attrition_split,
  metrics = metric_set(roc_auc, accuracy, sensitivity, specificity)
)

#Print the final performance metrics.
collect_metrics(ridge_final_fit)

## # A tibble: 4 x 4
##   .metric      .estimator .estimate .config
##   <chr>       <chr>      <dbl> <chr>
```



```
## 1 accuracy      binary      0.780 pre0_mod0_post0
## 2 sensitivity   binary      0.729 pre0_mod0_post0
## 3 specificity    binary      0.789 pre0_mod0_post0
## 4 roc_auc       binary      0.810 pre0_mod0_post0
```

#Generate a confusion matrix from the test predictions. A confusion matrix breaks down exactly where the

```
collect_predictions(ridge_final_fit) %>%
  conf_mat(truth = attrition, estimate = .pred_class)
```

```
##           Truth
## Prediction Yes  No
##           Yes 35 52
##           No 13 195
```

g. Run model on all the Data after assigning final hyper parameters.

#Fit the finalized workflow to the complete, unsplit dataset (df).

```
ridge_fit <- fit(ridge_final, data = df)
```

#View the final coefficients of the deployment model

```
tidy(ridge_fit)
```

```
## Warning: package 'glmnet' was built under R version 4.5.2
```

```
##
```

```
## Attaching package: 'Matrix'
```

```
## The following objects are masked from 'package:tidyr':
```

```
##
```

```
##      expand, pack, unpack
```

```
## Loaded glmnet 5.0
```

```
## # A tibble: 46 x 3
```

##	term	estimate	penalty
##	<chr>	<dbl>	<dbl>
## 1	(Intercept)	0.880	0.00001
## 2	age	0.205	0.00001
## 3	daily_rate	0.147	0.00001
## 4	distance_from_home	-0.244	0.00001
## 5	education	-0.0429	0.00001
## 6	employee_number	0.0648	0.00001
## 7	environment_satisfaction	0.390	0.00001
## 8	hourly_rate	-0.0139	0.00001
## 9	job_involvement	0.326	0.00001
## 10	job_level	0.0780	0.00001

```
## # i 36 more rows
```